

PES UNIVERSITY, BANGALORE
Department of Computer Science and Engineering

SOFTWARE ENGINEERING – MINI PROJECT

Project: SmartSpend — Smart Expense & Budget Management Application

Software Architecture & Design Specification

for SmartSpend (IEEE 1016 Format)
Version 1.0

Team H5:

Sriram S – PES1UG24CS469

Sriram K – PES1UG24CS468

Srijan Kumar – PES1UG24CS467

Sri Pranav Gautam Buduguru – PES1UG24CS466

Table of Contents

Part A — Architecture:

1. Introduction
2. Architectural Goals & Constraints
3. Component Diagram & Description
4. Architectural Pattern
5. Security Architecture
6. Requirements Traceability

Part B — Design:

7. UML Sequence Diagrams
8. API Design
9. Error Handling
10. Database Design
11. Design Constraints & Rationale

Part A — Software Architecture

1. Introduction

1.1 Purpose

This document specifies the software architecture and detailed design for SmartSpend, Team H5's expense and budget management application. It follows the standard IEEE 1016-style Software Design Description structure, split into an Architecture section (system-level structure and patterns) and a Design section (component-internal design: sequences, APIs, error handling, and data).

1.2 Scope

This document covers the complete backend and frontend architecture of SmartSpend as a single, self-contained web application. It does not cover deployment infrastructure beyond containerization (Docker), which is addressed at a high level only.

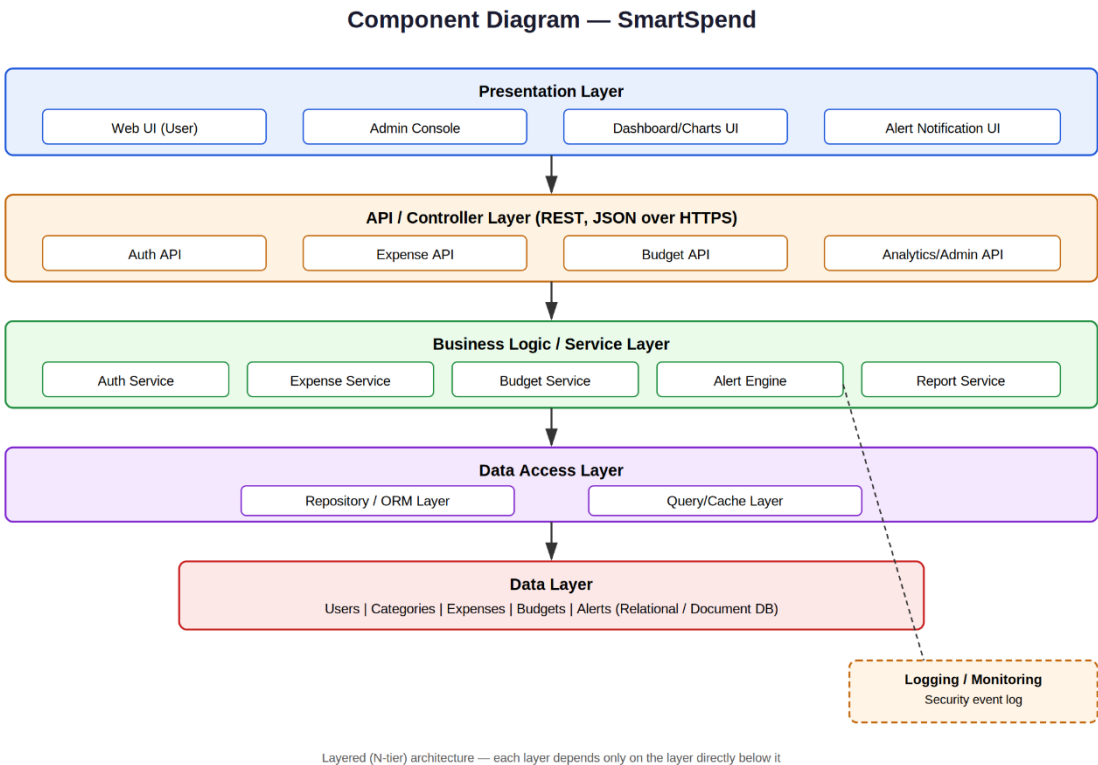
1.3 References

- SmartSpend Software Requirements Specification (SES-2026-H5_SRS.docx), Version 1.0.
- PES University Jackfruit Mini-Project Software Engineering Project Guidelines (UE24CS341A), 2026.
- IEEE Std 1016-2009 — IEEE Standard for Information Technology — Systems Design — Software Design Descriptions.

2. Architectural Goals & Constraints

- Modularity: each functional area (auth, expenses, budgets, alerts, reports, admin) must be independently developable and testable, since each is owned by a different team member (see SRS team ownership table).
- Security by design: authentication and authorization must be enforced at the API layer, not left to the client (traces to SEC-002).
- Technology constraint: must be implemented in either Python/Django or JavaScript (MERN stack), per the assigned project brief.
- Testability: business logic must be separable from the web framework so it can be unit tested with mocked dependencies (traces to NFR quality attributes in the SRS).

3. Component Diagram & Description



3.1 Component Descriptions

Component	Layer	Responsibility	Owner
Web UI / Admin Console / Dashboard	Presentation	Render screens, capture user input, display alerts and analytics	All (shared frontend)
Auth API + Auth Service	API / Business Logic	Registration, login, session/token issuance and validation	Member 1
Expense API + Expense Service	API / Business Logic	Create/edit/delete expenses, enforce validation rules (FR-005–FR-008)	Member 2
Budget API + Budget Service	API / Business Logic	Create/update budgets, recalculate cumulative spend (FR-009–FR-012)	Member 3
Alert Engine	Business Logic	Evaluate budget thresholds and generate alerts (FR-013–FR-015)	Member 3
Report Service + Analytics/Admin API	Business Logic / API	Aggregate spend data, generate analytics and exports (FR-016–FR-020)	Member 4
Repository / ORM Layer	Data Access	Abstracts all database access behind a single interface per entity	Shared
Database (Users, Categories, Expenses, Budgets, Alerts)	Data	Persistent storage	Shared
Logging / Monitoring	Cross-cutting	Captures security events (SEC-005) and operational logs	Member 4 (secondary)

4. Architectural Pattern

SmartSpend follows a layered (N-tier) architecture with five horizontal layers: Presentation, API/Controller, Business Logic (Service), Data Access, and Data. Each layer depends only on the layer directly beneath it, and no layer is permitted to skip a layer (e.g., the Presentation layer never queries the Database layer directly). This pattern was chosen over alternatives (e.g., microservices) because:

- The system's scope (single team, one semester, 8 core features) does not justify the operational overhead of independently deployed services.
- A layered architecture directly supports the modularity and testability goals in Section 2, since each service class can be unit tested with the Repository layer mocked out.
- It maps cleanly onto both candidate technology stacks (Django's MVT structure or an Express/Node MERN structure), keeping the architecture stack-agnostic at this stage of the project.

Within the Business Logic layer, each service additionally follows the Single Responsibility Principle — one service class per bounded functional area (Auth, Expense, Budget, Alert, Report) — which also matches the team's functional-ownership split, so that a given member's Git activity maps cleanly onto a given service.

5. Security Architecture

Security is enforced at two points in the architecture: (1) the API/Controller layer, where every request is authenticated (valid session token, SEC-004) and authorized (role check against the requested endpoint, SEC-002) before being forwarded to the Business Logic layer; and (2) the Business Logic layer itself, where object-level authorization is enforced (e.g., Expense Service verifies that the requesting user owns the expense being edited, per FR-006) so that authorization is never solely a UI-level concern.

Security Concern	Architectural Control	Traces To
Unauthenticated access	API layer rejects any request without a valid session token	SEC-004
Privilege escalation	API layer enforces RBAC; Admin endpoints reject User-role tokens	SEC-002
Credential exposure	Auth Service hashes passwords before persistence; hash never leaves the service boundary	SEC-001, SEC-006
Input-based attacks (e.g., injection)	All input validated/sanitized at the Service layer before reaching the Repository/ORM layer	SEC-003
Undetected intrusion attempts	Logging/Monitoring component records failed logins and unauthorized access attempts	SEC-005

6. Requirements Traceability (Architecture Level)

Each architectural component traces to the SRS functional area it implements, providing the Architecture Ref column referenced in the SRS's Appendix C (RTM):

Component	SRS Requirements Covered
Auth Service	FR-001–FR-004, SEC-001, SEC-004, SEC-005

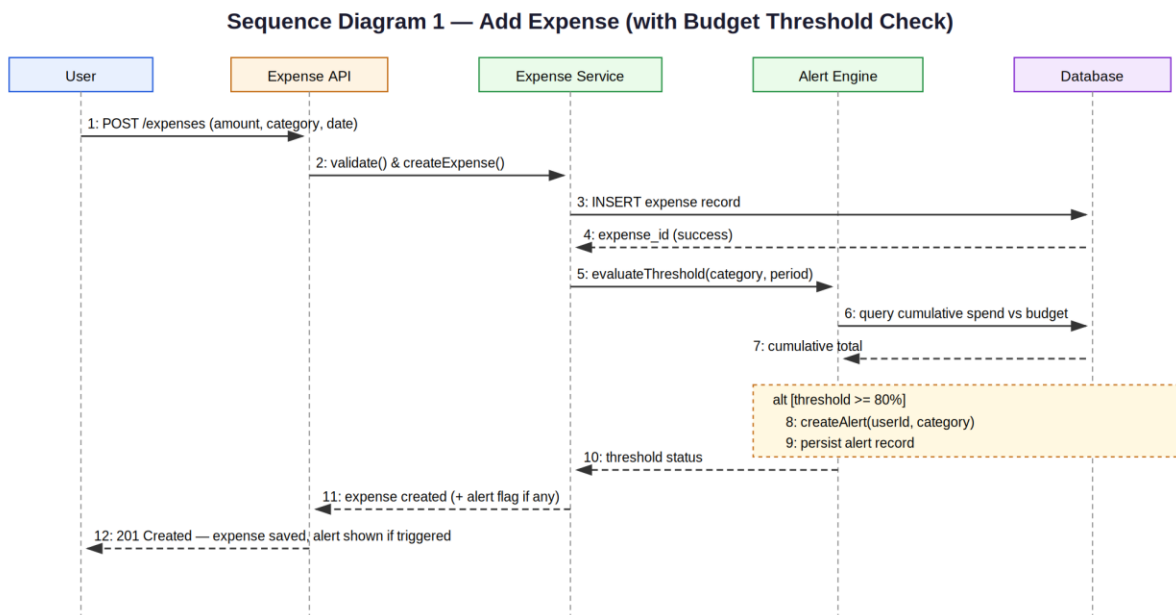
Component	SRS Requirements Covered
Expense Service	FR-005–FR-008
Budget Service	FR-009–FR-012
Alert Engine	FR-013–FR-015, NFR-002
Report Service	FR-016–FR-018
Admin/Analytics API	FR-019–FR-021, SEC-002
Full stack (all layers)	NFR-001 (response time under load)

Part B — Software Design

7. UML Sequence Diagrams

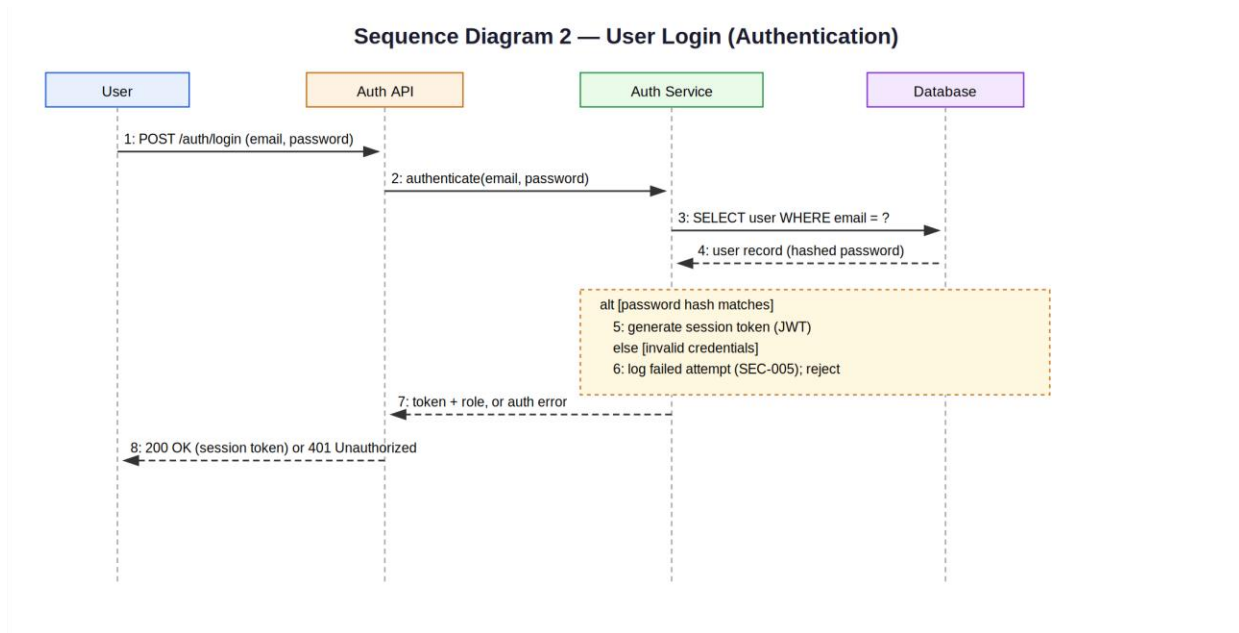
7.1 Sequence Diagram 1 — Add Expense (with Budget Threshold Check)

This sequence covers FR-005 (add expense), FR-008 (validation), FR-011 (real-time threshold recalculation), and FR-013/FR-014 (alert generation), corresponding to the Stimulus/Response Sequence documented in SRS Section 5.7.



7.2 Sequence Diagram 2 — User Login (Authentication)

This sequence covers FR-003 (authentication), FR-004 (session token issuance), SEC-001 (password hash comparison), and SEC-005 (logging of failed attempts).



8. API Design

All endpoints are versioned under `/api/v1` and communicate via JSON over HTTPS. Every endpoint other than registration and login requires a valid session token in the Authorization header (Bearer scheme).

Method	Endpoint	Purpose	Auth Required	Traces To
POST	<code>/api/v1/auth/register</code>	Create a new user account	No	FR-001, FR-002
POST	<code>/api/v1/auth/login</code>	Authenticate and issue a session token	No	FR-003, FR-004
POST	<code>/api/v1/expenses</code>	Create a new expense record	Yes (User)	FR-005, FR-008
PUT	<code>/api/v1/expenses/{id}</code>	Update an owned expense record	Yes (User, owner)	FR-006
DELETE	<code>/api/v1/expenses/{id}</code>	Delete an owned expense record	Yes (User, owner)	FR-006
POST	<code>/api/v1/budgets</code>	Create a new budget	Yes (User)	FR-009, FR-012
PUT	<code>/api/v1/budgets/{id}</code>	Update an owned budget	Yes (User, owner)	FR-010
GET	<code>/api/v1/alerts</code>	Retrieve active alerts for the logged-in user	Yes (User)	FR-015
GET	<code>/api/v1/analytics?from&to</code>	Retrieve category breakdown and trend data	Yes (User)	FR-016, FR-017
GET	<code>/api/v1/reports/export?from&to</code>	Export a CSV summary report	Yes (User)	FR-018
POST	<code>/api/v1/admin/categories</code>	Create/update a master category	Yes (Admin only)	FR-019, SEC-002
GET	<code>/api/v1/admin/analytics</code>	Retrieve aggregate system analytics	Yes (Admin only)	FR-020, SEC-002

8.1 API Contract Rules

- Every endpoint defines an explicit request schema (required vs. optional fields) and response schema, including error response shape.
- HTTP status codes are used consistently: 200/201 for success, 400 for validation errors, 401 for missing/invalid authentication, 403 for authorization failures, 404 for missing resources, 500 for unhandled server errors.
- An API contract, once other modules depend on it, is not changed silently; a breaking change requires a version increment or team-wide coordination.
- All monetary amounts are transmitted as decimal-safe values (never floating-point) to avoid rounding errors in budget calculations.

9. Error Handling

SmartSpend uses a consistent, layered error-handling strategy so that failures are never silent (per SRS Section 6.2) and never leak internal implementation detail to the client.

Layer	Error Handling Responsibility
API/Controller	Catches malformed requests and validation failures; maps internal exceptions to the correct HTTP status code and a client-safe error message (never a raw stack trace).
Business Logic (Service)	Raises typed, domain-specific exceptions (e.g., <code>InvalidAmountError</code> , <code>DuplicateBudgetPeriodError</code> , <code>UnauthorizedActionError</code>) rather than generic exceptions, so the API layer can map each to a precise status code.
Data Access (Repository)	Catches database-level exceptions (e.g., connection failure, constraint violation) and re-raises them as a generic <code>PersistenceError</code> , which the Service layer surfaces as a 500 response with a retry-safe message rather than exposing raw DB error text.
Alert Engine	If threshold evaluation fails, the expense save itself must still succeed (per NFR reliability goal); the failure is logged and the alert check is retried on the next expense write rather than blocking the user's primary action.

9.1 Representative Error Scenarios

Scenario	Handling	Response
Expense amount ≤ 0	Rejected at Service layer (FR-008)	400 Bad Request, field-level error
Duplicate email on registration	Rejected at Service layer (FR-002)	409 Conflict
Expired/invalid session token	Rejected at API layer (SEC-004)	401 Unauthorized
User calls Admin-only endpoint	Rejected at API layer (SEC-002)	403 Forbidden
Overlapping budget period	Rejected at Service layer (FR-012)	409 Conflict
Database temporarily unavailable	Caught at Repository layer, surfaced generically	503 Service Unavailable, retry-safe message

10. Database Design

The core entities and their key fields are summarized below (full field layout for Expense appears in the SRS, Appendix B):

Entity	Key Fields	Relationships
User	user_id, email, password_hash, role	1–many with Expense, Budget
Category	category_id, name, is_active	1–many with Expense, Budget
Expense	expense_id, user_id, category_id, amount, expense_date, note	many–1 with User, Category
Budget	budget_id, user_id, category_id, limit_amount, period, start_date	many–1 with User, Category
Alert	alert_id, user_id, budget_id, threshold_type, created_at, acknowledged	many–1 with User, Budget

11. Design Constraints & Rationale

- The Repository/ORM abstraction (Section 3) exists specifically so unit tests can substitute an in-memory or mocked repository, supporting the SRS's testability quality attribute without requiring a live database in CI.
- Alert generation (Section 7.1) is deliberately decoupled from the expense-save transaction's success path, so a failure in alert evaluation can never cause a valid expense to be rejected — this directly supports the SRS reliability requirement.
- The API is versioned (/api/v1) from the outset so that Phase 2+ iterations can introduce breaking changes behind a new version path without affecting already-integrated frontend code.